

STM32多端口GPIO流水灯寄存器实现

STM32多端口GPIO流水灯寄存器实现

- 0 前言
- 1 STM32寄存器底层逻辑
 - 1.1什么是STM32 GPIO
 - 1.2寄存器映射
 - 1.3外设时钟
- 2 基于寄存器方式实现流水灯程序编写
 - 2.1点亮流水灯
 - 2.2程序编写
- 3 硬件接线与实验现象说明
- 4 总结

0 前言

作为嵌入式新手，本次实验没有使用HAL库，可直接查阅STM32参考手册，通过寄存器地址操作GPIO。实验过程中会遇到外设时钟、掩码等诸多问题。本文记录入门概念、硬件接线、代码，以及同学们在操作过程的踩坑点，加深对寄存器映射和GPIO的理解。

1 STM32寄存器底层逻辑

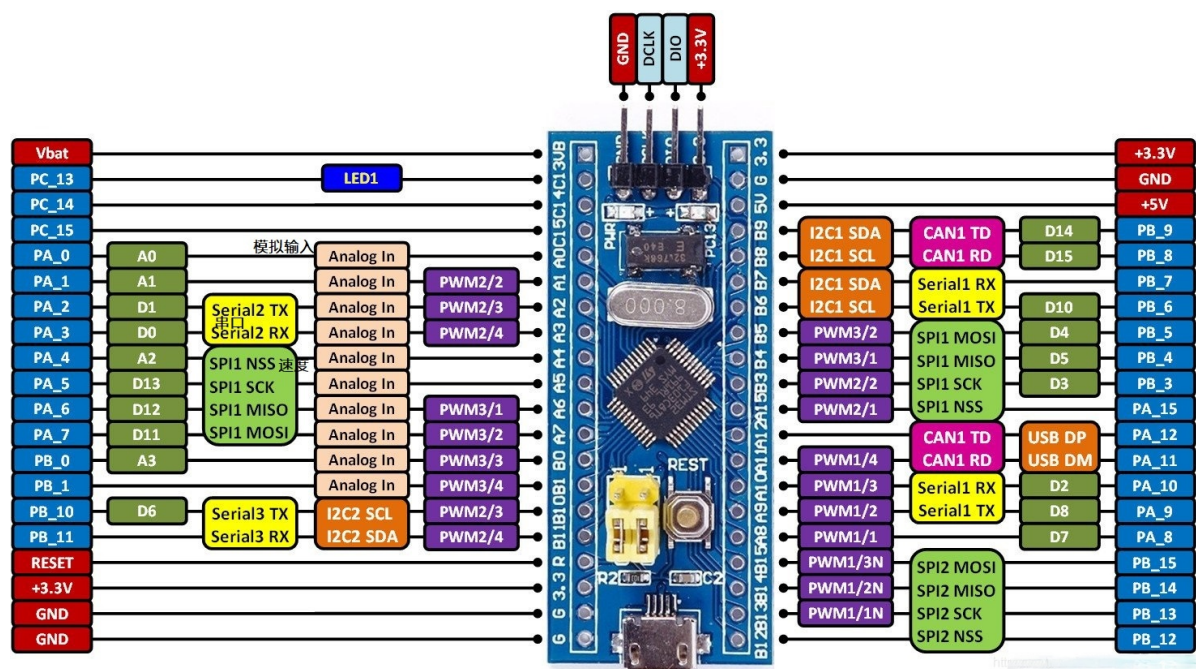
1.1什么是STM32 GPIO

基于电路的知识来说，GPIO叫通用输入输出引脚，是STM32单片机的基础外设，外设是集成在芯片内部独立硬件功能模块，是芯片CPU与外界的桥梁。GPIO可以读取外部的电平，也可以输出高低电平。

STM32的GPIO总共有8种工作模式：输入浮空、输入上拉、输入下拉、模拟输入、开漏输出、推挽式输出、推挽式复用功能、开漏复用功能。

本次流水灯实验选用的是**通用推挽输出模式**，须把GPIO配置成输出模式，通过输出高、低电平来控制LED灯的亮灭。输入模式则可以添加按键开开关。

STM32把GPIO引脚分成了GPIOA、GPIOB、GPIOC三个组，每一组下面最多有16个引脚，命名为Px0-Px15。详细引脚图如下：



1.2寄存器映射

这里我要说到存储器映射和寄存器映射。每个外设存在多个寄存器，每个寄存器都分配了唯一的寄存器地址。芯片厂商需要为各个片上外设分配内存地址，作为外设的**基地址**区间，这个过程叫做**存储器映射**。而**寄存器映射**是在此基础上利用外设基地址加上寄存器**偏移地址**，基地址与偏移地址组合为**寄存器地址**。

依靠寄存器映射，把硬件寄存器地址映射到内存地址空间。CPU通过这个地址，找到对应的寄存器，完成读写操作，以此控制外设。

举个例子，GPIOA 有自己的基地址0x40010800，再加上 0x0C 偏移，就得到 GPIOA_ODR 输出寄存器的地址。

1.3外设时钟

STM32上电之后，时钟默认关闭，需要先在寄存器RCC_APB2ENR把对应端口组别的时钟打开才能操作GPIO。

●注意：时钟是按**整个端口组**开启的。GPIOA时钟，是PA0-PA15全部引脚获得时钟。只有开启时钟，配置GPIO寄存器才会生效，LED灯实现亮灭。

在第2节我会详细写出如何查找外设寄存器地址。

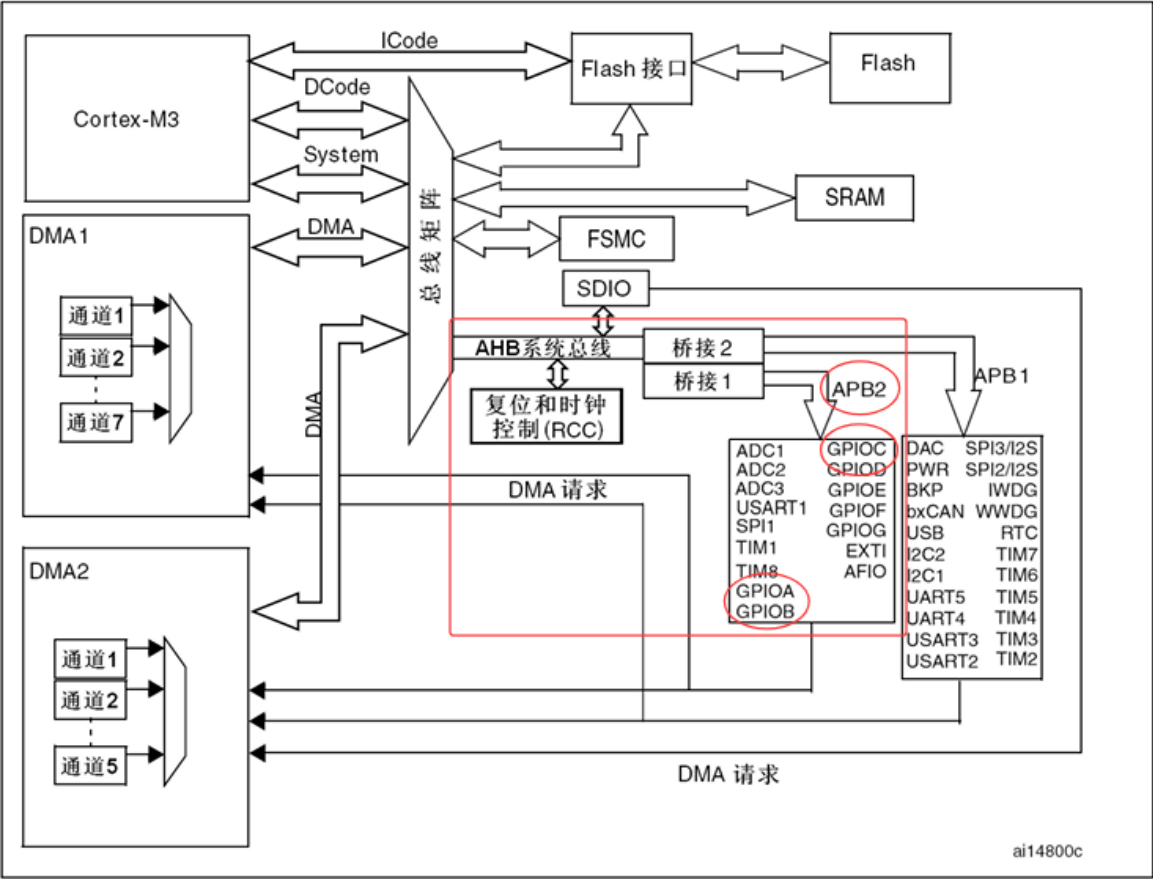
2 基于寄存器方式实现流水灯程序编写

2.1点亮流水灯

我们主要归结为以下几个步骤：

- ◆打开GPIO外设时钟

图1 系统结构



可以看出，总线矩阵连接到RCC，APB2 总线接GPIOA、GPIOB、GPIOC。RCC 管理外设时钟，向GPIO提供时钟信号。所以我们来查阅STM32中文参考手册得到他们的寄存器地址。

外设时钟寄存器地址

2.3 存储器映像

请参考相应器件的数据手册中的存储器映像图。表1列出了所用STM32F10xxx中内置外设的起始地址。

表1 寄存器组起始地址

起始地址	外设	总线	寄存器映像
0x5000 0000 – 0x5003 FFFF	USB OTG 全速	AHB	参见26.14.6节
0x4003 0000 – 0x4FFF FFFF	保留		
0x4002 8000 – 0x4002 9FFF	以太网		参见27.8.5节
0x4002 3400 - 0x4002 3FFF	保留		
0x4002 3000 - 0x4002 33FF	CRC	AHB	参见3.4.4节
0x4002 2000 - 0x4002 23FF	闪存存储器接口		
0x4002 1400 - 0x4002 1FFF	保留		
0x4002 1000 - 0x4002 13FF	复位和时钟控制(RCC)		参见6.3.11节
0x4002 0800 - 0x4002 0FFF	保留	AHB	
0x4002 0400 - 0x4002 07FF	DMA2		参见10.4.7节

可知RCC的基地址为0x40021000。

6.3.7 APB2 外设时钟使能寄存器(RCC_APB2ENR)

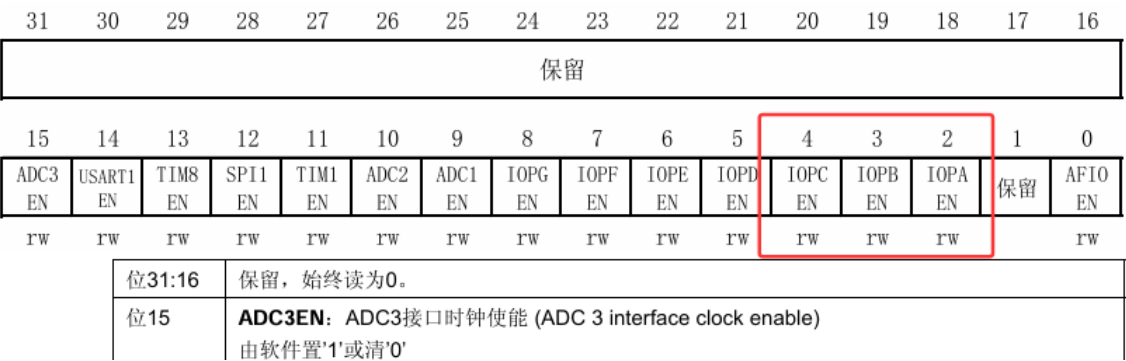
偏移地址：0x18

复位值：0x0000 0000

访问：字，半字和字节访问

通常无访问等待周期。但在APB2总线上的外设被访问时，将插入等待状态直到APB2的外设访问结束。

注： 当外设时钟没有启用时，软件不能读出外设寄存器的数值，返回的数值始终是0x0。



RCC的偏移地址为0x18，RCC基地址0x40021000加上偏移地址0x18构成寄存器地址0x40021018，就可以定位RCC内寄存器。

打开外部时钟

位2、位3、位4、分别是GPIOA、GPIOB、GPIOC的端口时钟，对应位写1，为打开时钟，写 0 关闭。代码编写如下：

```
1 //宏定义寄存器地址方便编程
2 #define RCC_APB2ENR (*(unsigned int *)0x40021018)
3 //开启GPIOA（数字1左移两位得到00000100位2）、GPIOB(位3)、GPIOC(位4)时钟
4 RCC_APB2ENR |= (1<<2) | (1<<3) | (1<<4);
```

对应GPIOA、GPIOB、GPIOC时钟打开。

◆ GPIO寄存器地址,配置GPIO推挽输出

本实验我设置的是PA0、PB1、PC14三个引脚：

0x4001 1400 - 0x4001 17FF	GPIO端口D	参见8.5节
0x4001 1000 - 0x4001 13FF	GPIO端口C	参见8.5节
0x4001 0C00 - 0x4001 0FFF	GPIO端口B	参见8.5节
0x4001 0800 - 0x4001 0BFF	GPIO端口A	参见8.5节
0x4001 0400 - 0x4001 07FF	EXTI	参见9.3.7节
0x4001 0000 - 0x4001 03FF	AFIO	参见8.5节

可以看到A、B、C三类端口基地址。

8.2.1 端口配置低寄存器(GPIOx_CRL) (x=A..E)

偏移地址: 0x00

复位值: 0x4444 4444

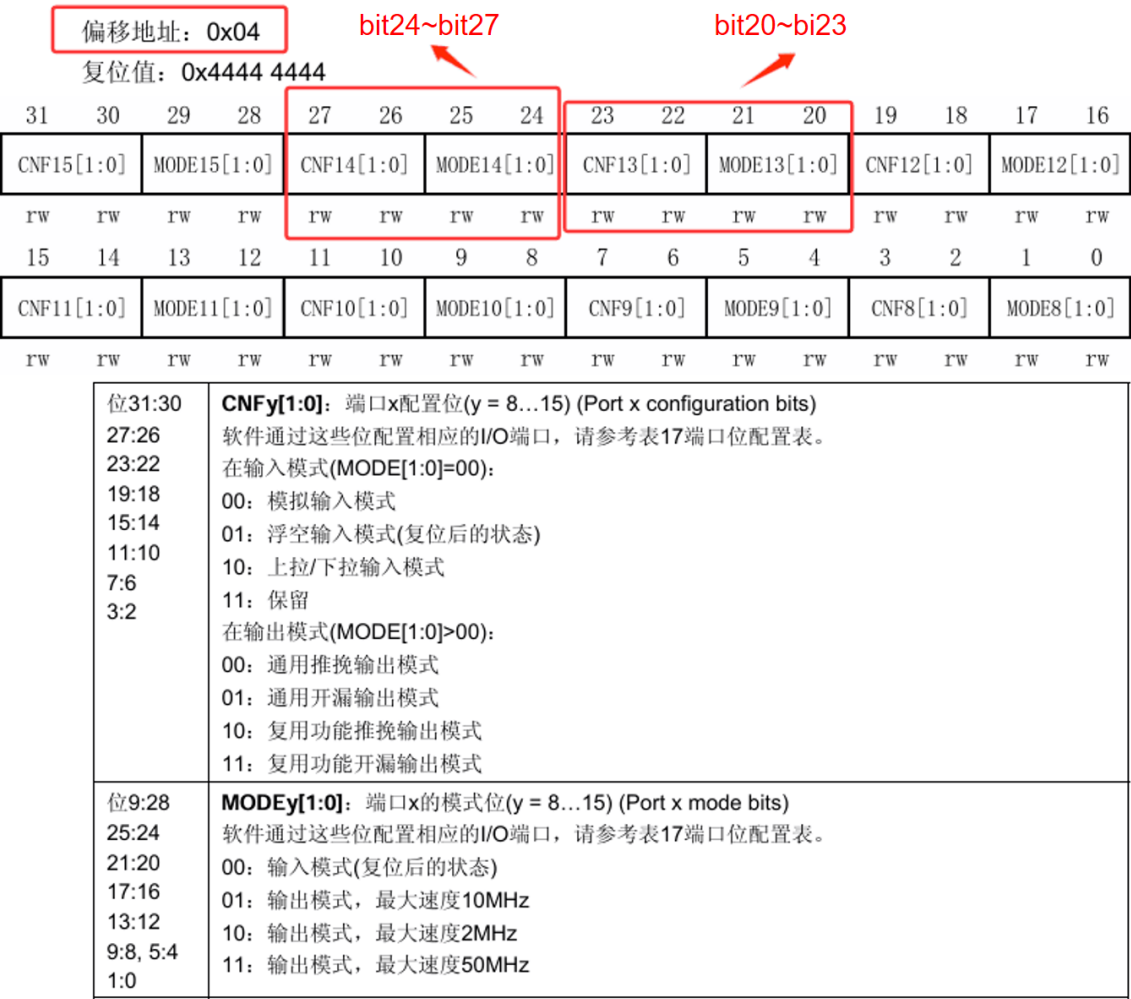
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNF7[1:0]		MODE7[1:0]		CNF6[1:0]		MODE6[1:0]		CNF5[1:0]		MODE5[1:0]		CNF4[1:0]		MODE4[1:0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNF3[1:0]		MODE3[1:0]		CNF2[1:0]		MODE2[1:0]		CNF1[1:0]		MODE1[1:0]		CNF0[1:0]		MODE0[1:0]	
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW

位31:30	CNFy[1:0]: 端口x配置位(y = 0...7) (Port x configuration bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 在输入模式(MODE[1:0]=00): 00: 模拟输入模式 01: 浮空输入模式(复位后的状态) 10: 上拉/下拉输入模式 11: 保留 在输出模式(MODE[1:0]>00): 00: 通用推挽输出模式 01: 通用开漏输出模式 10: 复用功能推挽输出模式 11: 复用功能开漏输出模式
27:26	
23:22	
19:18	
15:14	
11:10	
7:6	
3:2	
位29:28	MODEy[1:0]: 端口x的模式位(y = 0...7) (Port x mode bits) 软件通过这些位配置相应的I/O端口, 请参考表17端口位配置表。 00: 输入模式(复位后的状态) 01: 输出模式, 最大速度10MHz 10: 输出模式, 最大速度2MHz 11: 输出模式, 最大速度50MHz
25:24	
21:20	
17:16	
13:12	
9:8, 5:4	
1:0	

bit4~bit7

bit0~bit3

8.2.2 端口配置高寄存器(GPIOx_CRH) (x=A..E)



因为STM32为32bit处理器, 每个引脚控制四个bit, 故有8位引脚, 分高位和低位。当GPIO引脚为0-7, 如PA1-pA7,为低寄存器_CRL,偏移地址为0x00,高寄存器CRH偏移地址为0x04, 所以PA0为0x40010800, PB1为0x40010C00, PC13/14为0x40011004。

```
1 #define GPIOA_CRL      (*(unsigned int *)0x40010800)
2 #define GPIOB_CRL      (*(unsigned int *)0x40010C00)
3 #define GPIOC_CRH      (*(unsigned int *)0x40011004)
```

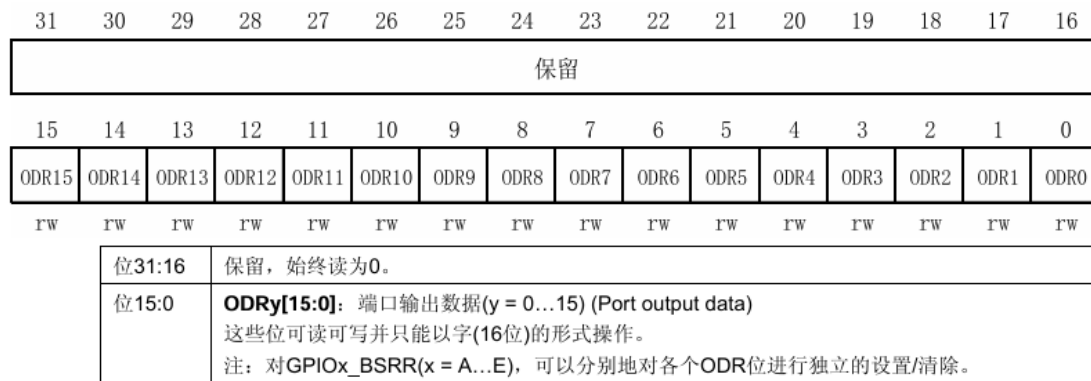
PA0控制bit0-3,设置最大速率2MHZ,配置码则为0x00000002, 通过掩码先清除4个配置位,再写入配置值,以此来配合灯推挽输出。

```
1 //----- PA0 CRL bit0-3推挽输出2MHz -----
2 GPIOA_CRL &= 0xFFFFFFF0;
3 GPIOA_CRL |= 0x00000002;
4
5 //----- PB1 CRL bit4-7推挽输出2MHz -----
6 GPIOB_CRL &= 0xFFFFF0F;
7 GPIOB_CRL |= 0x00000020;
8
9 //----- PC13 GPIOC_CRH bit20-23, 推挽输出2MHz -----
10 GPIOC_CRH &= 0xFF0FFFFF;
11 GPIOC_CRH |= 0x00200000;
12
13 //----- PC14 GPIOC_CRH bit24-27, 推挽输出2MHz -----
14 GPIOC_CRH &= 0xF0FFFFFF;
```

8.2.4 端口输出数据寄存器(GPIOx_ODR) (x=A..E)

地址偏移: 0Ch

复位值: 0x0000 0000



除了配置好GPIO工作模式, 还需要输出数据, 偏移地址为0C,所以我们将应交的后两位换成0C为寄存器ODR用来输出数据。

```
1 #define GPIOA_ODR    (*(unsigned int *)0x4001080C)
2 #define GPIOB_ODR    (*(unsigned int *)0x40010C0C)
3 #define GPIOC_ODR    (*(unsigned int *)0x4001100C)
```

◆ 低电平点亮LED灯

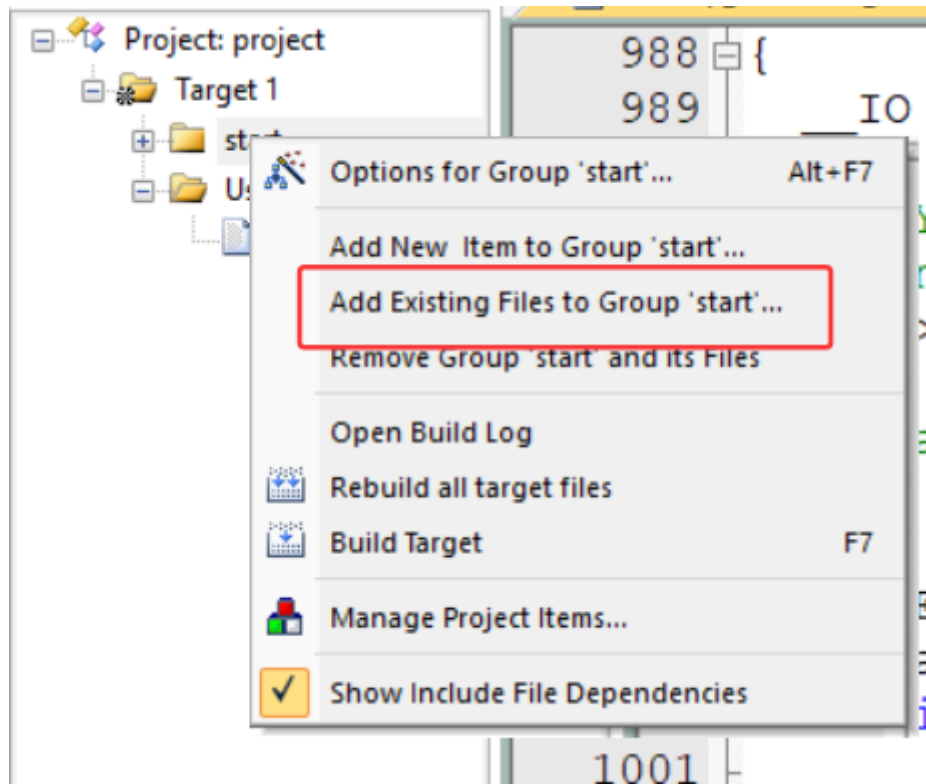
以点亮PA0为例, 将1移位到对应位上, 如0位: $1 \ll 0$, PB1: $1 \ll 1$, PC13: $1 \ll 13$. y因为低电平点亮, 在PA0引脚上取反。流水灯再添加while循环, 1s延时。

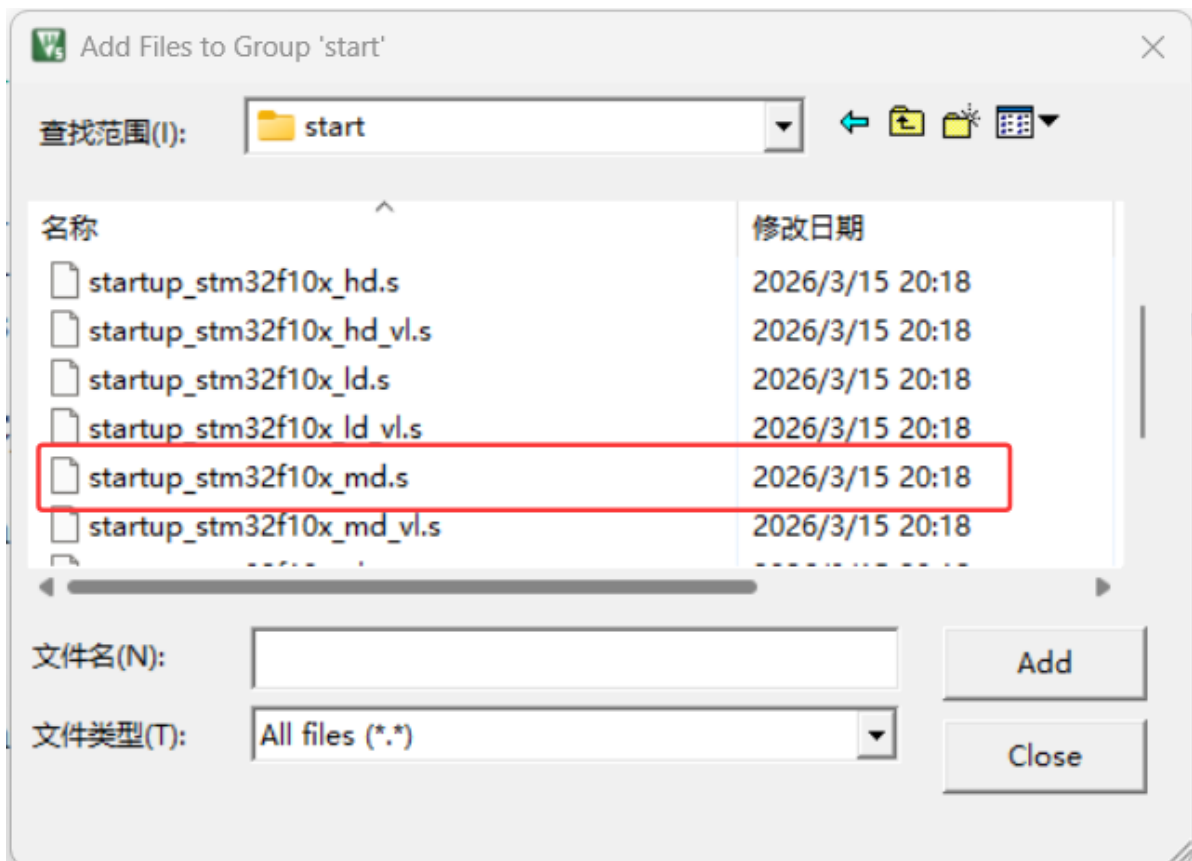
```
1 void delay_1s(void)
2 {
3     unsigned int i,j;
4     for(i=0;i<800;i++)
5         for(j=0;j<12000;j++);
6 }
7 while(1)
8 {
9     //1. 点亮PC14, PA0、PB1、PC13熄灭
10    GPIOA_ODR |= (1<<0);
11    GPIOB_ODR |= (1<<1);
12    GPIOC_ODR |= (1<<13);
13    GPIOC_ODR &= ~(1<<14);
14    delay_1s();
15
16    //2. 点亮PA0, PC14、PB1、PC13熄灭
17    GPIOA_ODR &= ~(1<<0);
18    GPIOB_ODR |= (1<<1);
19    GPIOC_ODR |= (1<<13) | (1<<14);
20    delay_1s();
21
22    //3. 点亮PB1, PC14、PA0、PC13熄灭
23    GPIOA_ODR |= (1<<0);
24    GPIOB_ODR &= ~(1<<1);
25    GPIOC_ODR |= (1<<13) | (1<<14);
26    delay_1s();
```

```
27
28 //4. 点亮PC13板载LED, PC14、PA0、PB1熄灭
29 GPIOA_ODR |= (1<<0);
30 GPIOB_ODR |= (1<<1);
31 GPIOC_ODR |= (1<<14);
32 GPIOC_ODR &= ~(1<<13);
33 delay_1s();
34 }
```

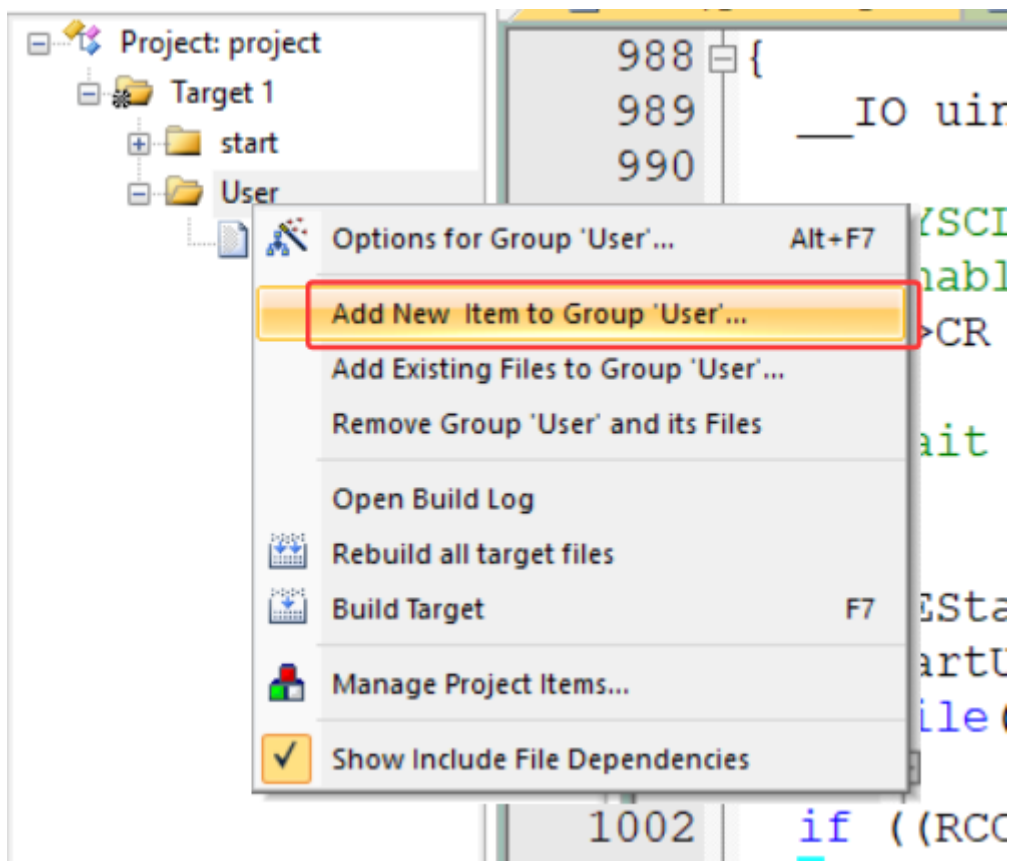
2.2程序编写

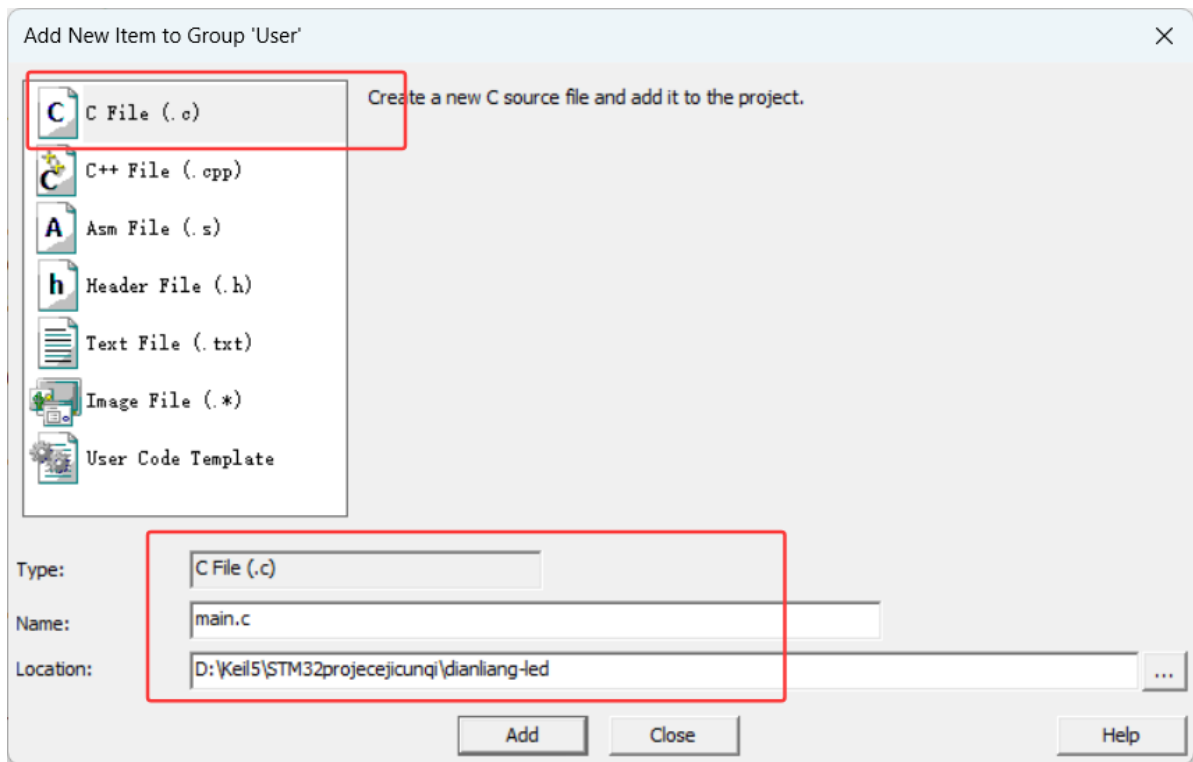
◆ 添加启动文件，他是stm32启动文件md.s是寄存器方式编写的必要文件，可以去ARM官网上查找。





◆ 其次创建main.c文件编写我们的主程序





● 注意，两个重要文件都需放在项目文件夹下。

由于上文已经详细说明各个代码模块编写思路，这里附上完整代码：

◆ 第一个实验为3个LED灯，轮流闪烁，间隔时长1秒。

```
1  typedef unsigned int  unsignedint;
2
3  #define RCC_APB2ENR  (*(unsigned int *)0x40021018)
4
5  #define GPIOA_CRL    (*(unsigned int *)0x40010800)
6  #define GPIOA_ODR    (*(unsigned int *)0x4001080C)
7
8  #define GPIOB_CRL    (*(unsigned int *)0x40010C00)
9  #define GPIOB_ODR    (*(unsigned int *)0x40010C0C)
10
11 #define GPIOC_CRH     (*(unsigned int *)0x40011004)
12 #define GPIOC_ODR     (*(unsigned int *)0x4001100C)
13
14 //软件近似1s延时
15 void delay_1s(void)
16 {
17     unsigned int i,j;
18     for(i=0;i<800;i++)
19         for(j=0;j<12000;j++);
20 }
21
22 void LED_GPIO_Init(void)
23 {
24     //开启GPIOA(bit2)、GPIOB(bit3)、GPIOC(bit4)时钟
25     RCC_APB2ENR |= (1<<2) | (1<<3) | (1<<4);
26
27     //----- PA0配置 CRL bit0-3 推挽输出 2MHz 0010(0x2) -----
28     GPIOA_CRL &= 0xFFFFFFFF; //清除PA0对应的4位
29     GPIOA_CRL |= 0x00000002;
```

```

30
31 //----- PB1配置 CRL bit4-7 推挽输出2MHz -----
32 GPIOB_CRL &= 0xFFFFF0F;
33 GPIOB_CRL |= 0x00000020;
34
35 //----- PC14配置 GPIOC_CRH bit24~27 推挽输出2MHz -----
36 GPIOC_CRH &= 0xF0FFFFFF;
37 GPIOC_CRH |= 0x02000000;
38
39 //全部LED初始熄灭，输出高电平（低电平点亮）
40 GPIOA_ODR |= (1<<0);
41 GPIOB_ODR |= (1<<1);
42 GPIOC_ODR |= (1<<14);
43 }
44
45 int main(void)
46 {
47     LED_GPIO_Init();
48     while(1)
49     {
50         //点亮PA0, PB1、PC14熄灭
51         GPIOA_ODR &= ~(1<<0);
52         GPIOB_ODR |= (1<<1);
53         GPIOC_ODR |= (1<<14);
54         delay_1s();
55
56         //点亮PB1, PA0、PC14熄灭
57         GPIOA_ODR |= (1<<0);
58         GPIOB_ODR &= ~(1<<1);
59         GPIOC_ODR |= (1<<14);
60         delay_1s();
61
62         //点亮PC14, PA0、PB1熄灭
63         GPIOA_ODR |= (1<<0);
64         GPIOB_ODR |= (1<<1);
65         GPIOC_ODR &= ~(1<<14);
66         delay_1s();
67     }
68 }

```

◆第二个实验需要把板载灯PC13加入流水灯中，，我需要把PC13也加入循环中，完整代码如下：

```

1  typedef unsigned int  unsignedint;
2
3  #define RCC_APB2ENR  (*(unsigned int *)0x40021018)
4
5  #define GPIOA_CRL    (*(unsigned int *)0x40010800)
6  #define GPIOA_ODR    (*(unsigned int *)0x4001080C)
7
8  #define GPIOB_CRL    (*(unsigned int *)0x40010C00)
9  #define GPIOB_ODR    (*(unsigned int *)0x40010C0C)
10
11 #define GPIOC_CRH    (*(unsigned int *)0x40011004)
12 #define GPIOC_ODR    (*(unsigned int *)0x4001100C)
13

```

```

14 //软件近似1s延时
15 void delay_1s(void)
16 {
17     unsigned int i,j;
18     for(i=0;i<800;i++)
19         for(j=0;j<12000u;j++);
20 }
21
22 void LED_GPIO_Init(void)
23 {
24     //开启GPIOA(bit2) 、GPIOB(bit3)、GPIOC(bit4)时钟
25     RCC_APB2ENR |= (1<<2) | (1<<3) | (1<<4);
26
27     //----- PA0 CRL bit0-3 推挽输出2MHz -----
28     GPIOA_CRL &= 0xFFFFFFFF;
29     GPIOA_CRL |= 0x00000002;
30
31     //----- PB1 CRL bit4-7 推挽输出2MHz -----
32     GPIOB_CRL &= 0xFFFFFFFF;
33     GPIOB_CRL |= 0x00000020;
34
35     //----- PC13 GPIOC_CRH bit20-23 备份域IO, 推挽输出2MHz -----
36     GPIOC_CRH &= 0xFF0FFFFF;
37     GPIOC_CRH |= 0x00200000;
38
39     //----- PC14 GPIOC_CRH bit24-27 备份域IO, 推挽输出2MHz -----
40     GPIOC_CRH &= 0xF0FFFFFF;
41     GPIOC_CRH |= 0x02000000;
42
43     //全部LED初始状态熄灭, 输出高电平
44     GPIOA_ODR |= (1<<0);
45     GPIOB_ODR |= (1<<1);
46     GPIOC_ODR |= (1<<13) | (1<<14);
47 }
48
49 int main(void)
50 {
51     LED_GPIO_Init();
52     while(1)
53     {
54         //1.点亮PC14, PA0、PB1、PC13熄灭
55         GPIOA_ODR |= (1<<0);
56         GPIOB_ODR |= (1<<1);
57         GPIOC_ODR |= (1<<13);
58         GPIOC_ODR &= ~(1<<14);
59         delay_1s();
60
61         //2.点亮PA0, PC14、PB1、PC13熄灭
62         GPIOA_ODR &= ~(1<<0);
63         GPIOB_ODR |= (1<<1);
64         GPIOC_ODR |= (1<<13) | (1<<14);
65         delay_1s();
66
67         //3.点亮PB1, PC14、PA0、PC13熄灭
68         GPIOA_ODR |= (1<<0);

```

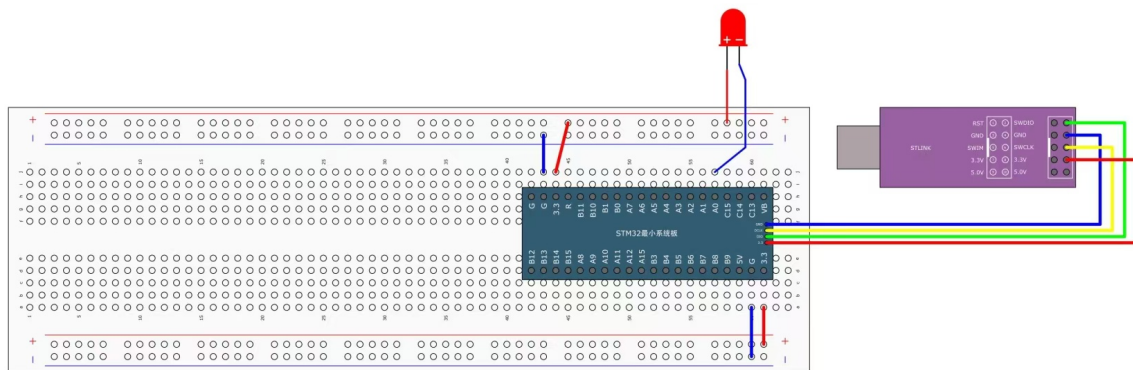
```

69     GPIOB_ODR &= ~(1<<1);
70     GPIOC_ODR |= (1<<13) | (1<<14);
71     delay_1s();
72
73     //4. 点亮PC13板载LED, PC14、PA0、PB1熄灭
74     GPIOA_ODR |= (1<<0);
75     GPIOB_ODR |= (1<<1);
76     GPIOC_ODR |= (1<<14);
77     GPIOC_ODR &= ~(1<<13);
78     delay_1s();
79 }
80 }

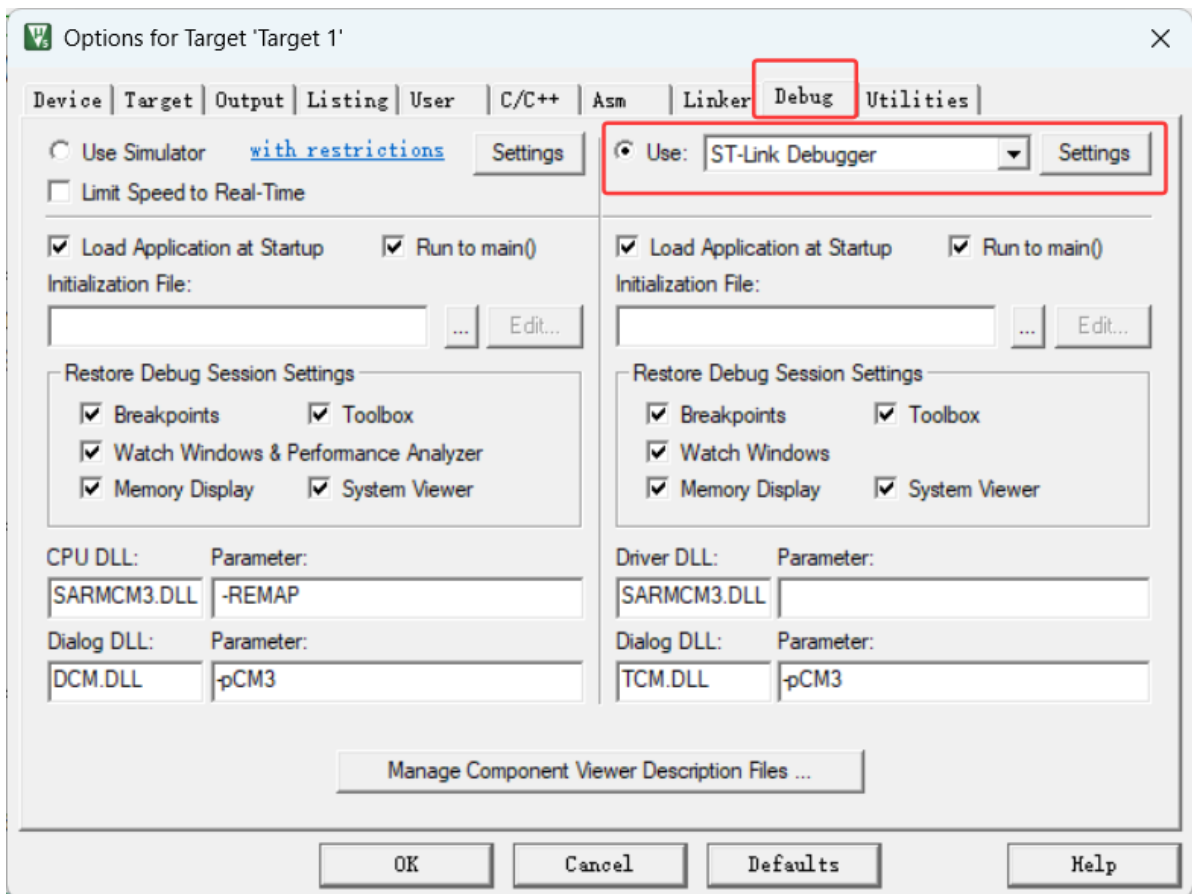
```

3 硬件接线与实验现象说明

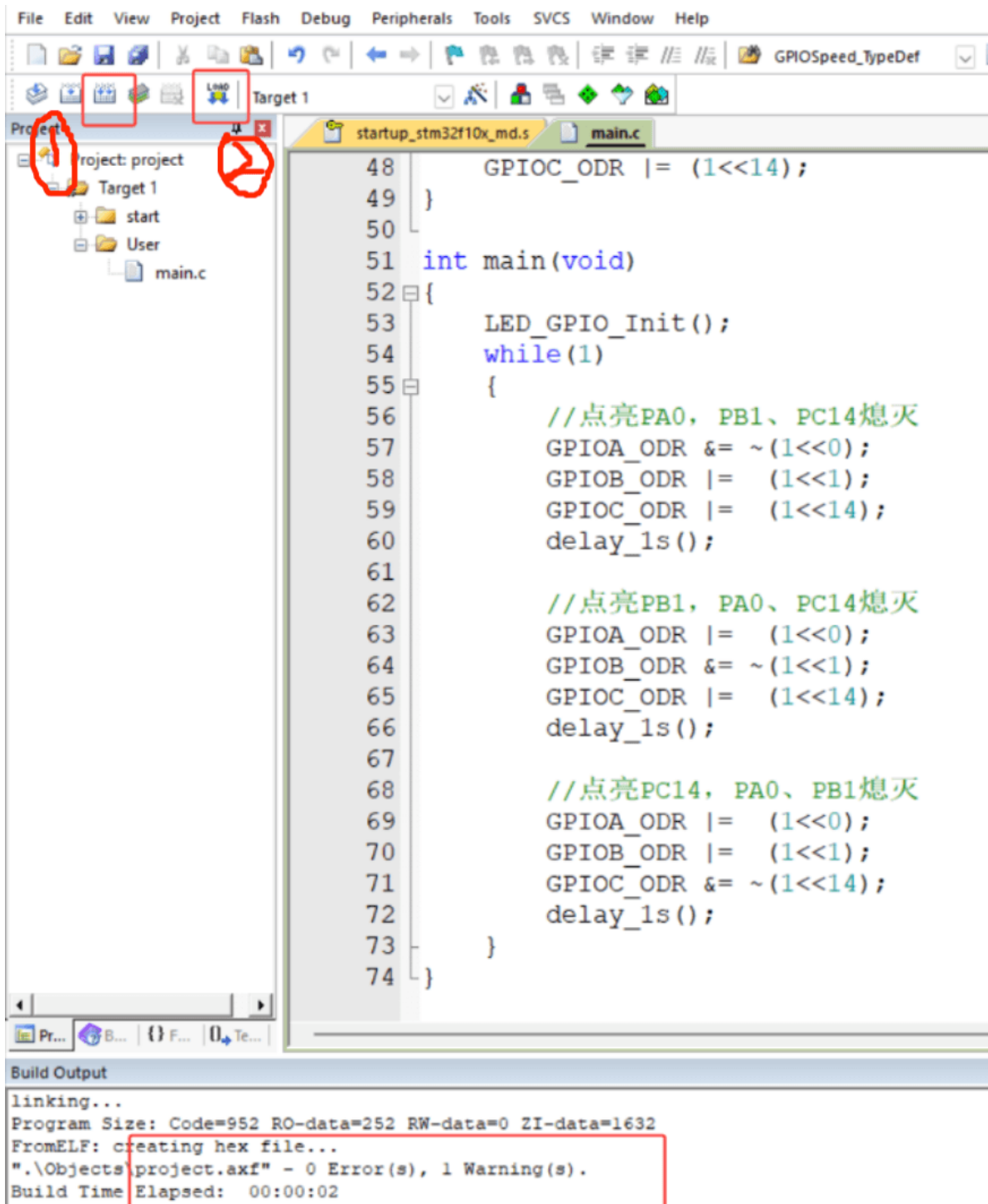
如图是简易接线图，其中我的蓝灯、红灯、绿灯分别放到了PA0、PB1、PC14引脚。



烧录我选择stlink下载。

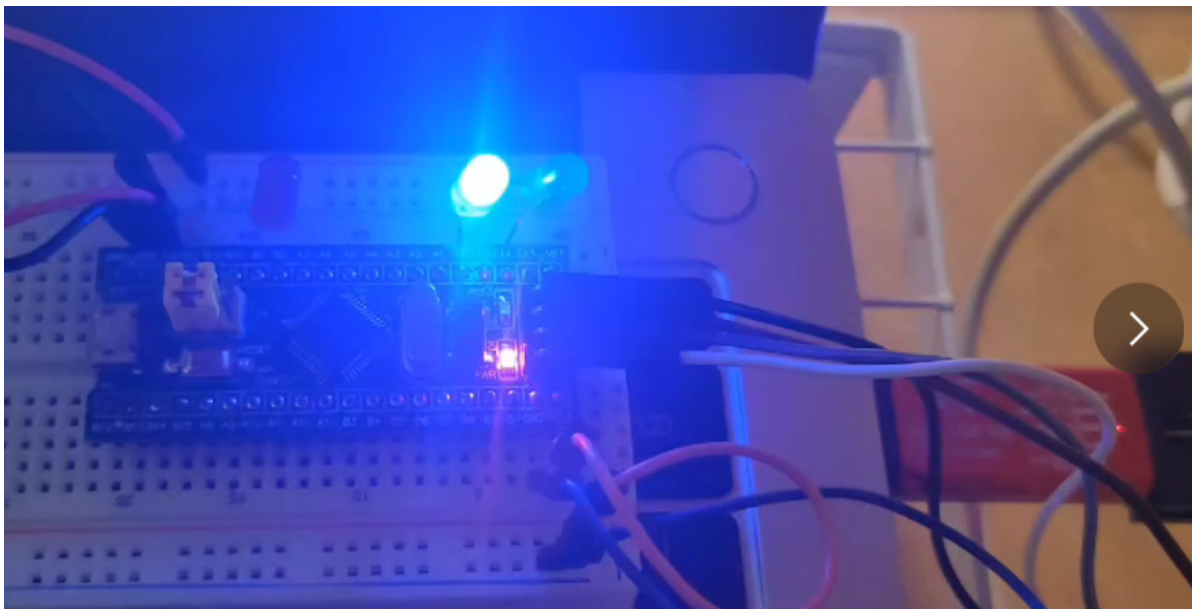


编译无错误后，再进行烧录。



◆第一个实验为3个LED灯，轮流闪烁，间隔时长1秒。

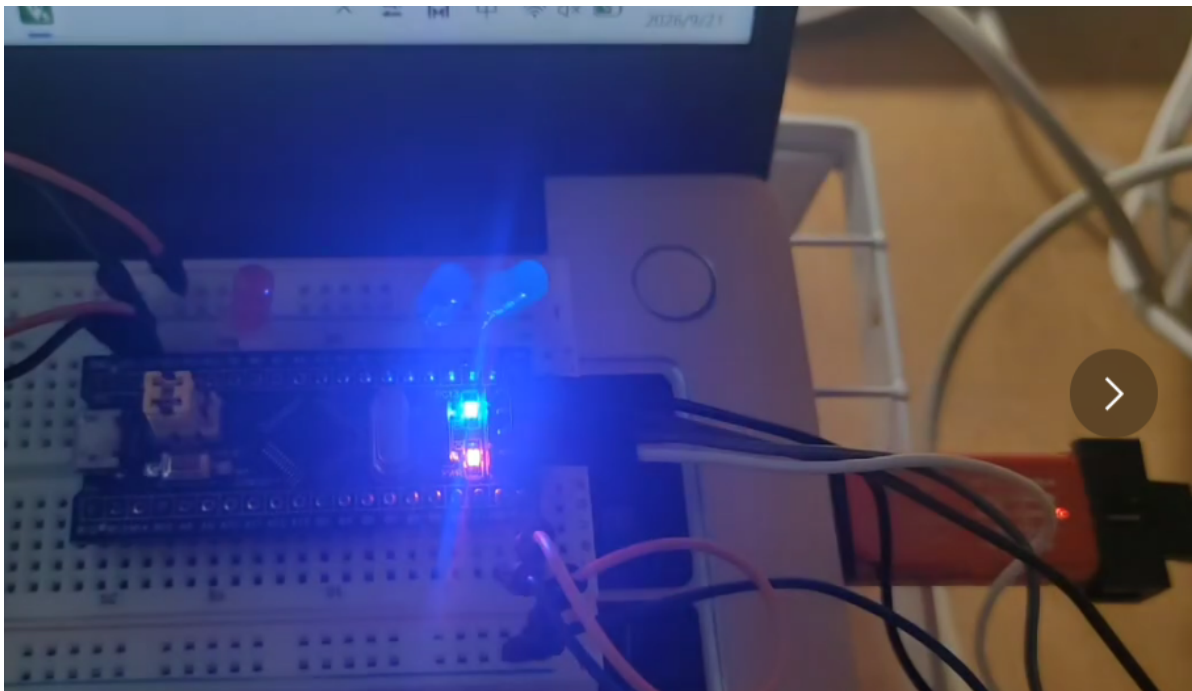
实验现象为红蓝绿三个灯轮流亮：



◆第二个实验需要把板载灯PC13加入流水灯中，我需要把PC13也加入循环中。

```
1  GPIOC_ODR |= (1<<13) | (1<<14);
2
3  //4. 点亮PC13, PC14, PA0, PB1熄灭
4  GPIOA_ODR |= (1<<0);
5  GPIOB_ODR |= (1<<1);
6  GPIOC_ODR |= (1<<14);
7  GPIOC_ODR &= ~(1<<13);
8  delay_1s();
9
```

实验现象如下：



可以看到PC13成功点亮。

4 总结

从寄存器方式实现流水灯，我更加深刻的理解到了寄存器的工作原理。首先需要了解位操作<<移位、|=赋值、&=与、~取反在单片机寄存器配置中的实际用途。再到写入时钟寄存器地址，可以向外设GPIO下达指令，RCC_APB2ENR开启时钟信号，CRL、CRH设置GPIO工作方式，每一个GPIO引脚占用连续4个bit位，ODR输出寄存器负责控制引脚输出高低电平等作用，读取地址了解引脚状态来驱动外部器件工作。该流水灯为低电平（LED3.3V与0V引脚电压差）点亮LED，因此将对应bit清零时灯点亮，置1时灯熄灭。其中，学会看懂查找板子手册也很重要，学习如何查找寄存器地址，映射到程序编写，引脚接线，再进行烧录，操作一个比较完整的项目，不断排查错误，锻炼了我的完整项目实操能力。